Lab 1:
Block Cipher: Encryption / decryption using Library

Course:
MECR1063 Cryptography Engineering

PART A: Encryption and decryption (binary ciphertext)	[20 *marks*]

A 1	Command-line:
	`echo "Keertennah Devi A/P Ponnambalam MEC254026" > plaintext2A-1.txt`
	
	Display the contents of the created file using a command-line command.
	
	Command-line:
	`cat plaintext2A-1.txt`

A 2	Command-line: `openssl enc -aes-256-ecb -e -in plaintext2A-1.txt -out ciphertext2A-1.bin -K 80000000000000000000000000000000000000000000000000000000000000001`

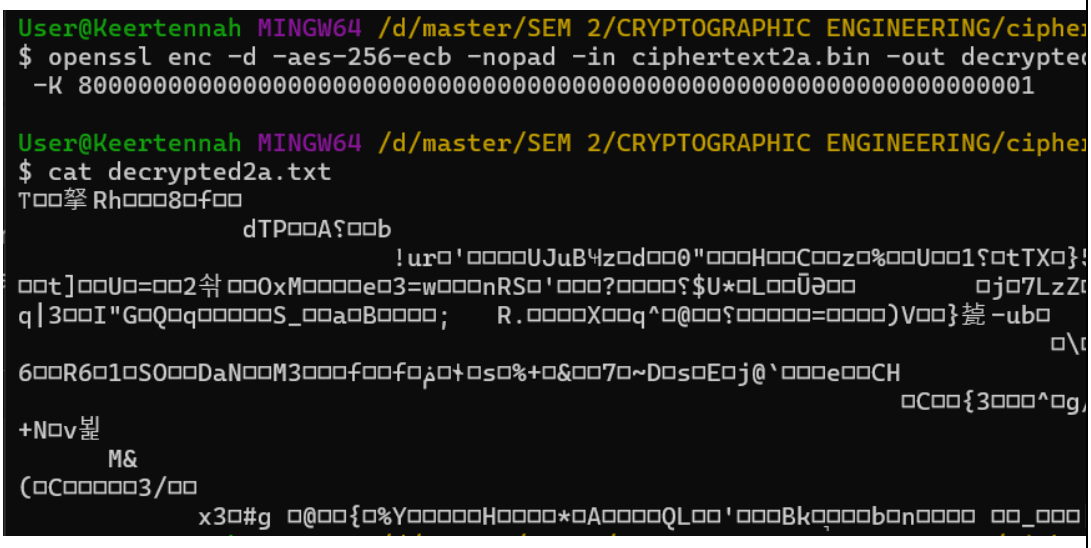
A 3	i) using `xxd`
	`xxd ciphertext2A-1.bin`
	
	ii) using `cat`
	`cat ciphertext2A-1.bin`
	
	iii) using `hexdump` (canonical format)
	`hexdump -C ciphertext2A-1.bin`
	
	Explain and conclude
	
	cat displays raw binary, usually outputting unreadable characters to the terminal. xxd creates a simple hex dump showing a hexadecimal offset and the hex values. hexdump -C provides a canonical format with offset, hex data in two columns, and ASCII representation. This is often the most suitable for analyzing ciphertext.

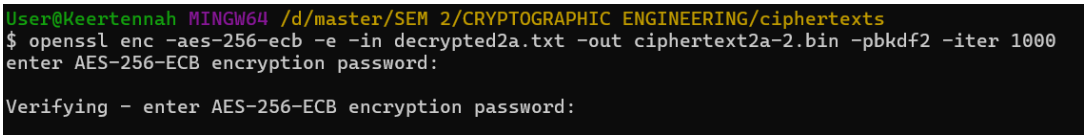
A 4	Command-line: `openssl enc -aes-256-ecb -d -in ciphertext2A-1.bin -out decrypted2A-1.txt -K 80000000000000000000000000000000000000000000000000000000000000001`
A 5	Command-line: `cat decrypted2A-1.txt`
A 6	Take a screenshot of the command lines used.

```
User@Keertennah MINGW64 ~
$ echo "Keertennah Devi A/P Ponnambalam MEC254026" > plaintext2A-1.txt

User@Keertennah MINGW64 ~
$ cat plaintext2A-1.txt
Keertennah Devi A/P Ponnambalam MEC254026

User@Keertennah MINGW64 ~
$ openssl enc -aes-256-ecb -e -in plaintext2A-1.txt -out ciphertext2A-1
 80000000000000000000000000000000000000000000000000000000000000001

User@Keertennah MINGW64 ~
$ xxd ciphertext2A-1.bin
00000000: d85a 19a8 2457 47db f37b ce21 7ea6 b4e9  .Z..$WG..{.!~...
00000010: c6a7 57f8 8ac7 35f1 7c0e 8743 8d76 3769  ..W...5.|..C.v7i
00000020: f7b9 936a a99c b6c8 365e 126d 8044 479e  ...j....6^.m.DG.

User@Keertennah MINGW64 ~
$ cat ciphertext2A-1.bin
□Z□$WG□□{□!~□□□2W□□□5□|□C□v7i□□□j□□□□6^m□DG□
User@Keertennah MINGW64 ~
$ hexdump -C ciphertext2A-1.bin
00000000: d85a 19a8 2457 47db f37b ce21 7ea6 b4e9  .Z..$WG..{.!~...
00000010: c6a7 57f8 8ac7 35f1 7c0e 8743 8d76 3769  ..W...5.|..C.v7i
00000020: f7b9 936a a99c b6c8 365e 126d 8044 479e  ...j....6^.m.DG.

User@Keertennah MINGW64 ~
$ openssl enc -aes-256-ecb -d -in ciphertext2A-1.bin -out decrypted2A-1
 80000000000000000000000000000000000000000000000000000000000000001

User@Keertennah MINGW64 ~
$ cat decrypted2A-1.txt
Keertennah Devi A/P Ponnambalam MEC254026
```

PART B: Decryption (binary ciphertext)	[5 *marks*]
B 1	Command-line: openssl enc -d -aes-256-ecb -nopad -in ciphertext2a.bin -out decrypted2a.txt -K 8000000000000000000000000000000000000000000000000000000000000001
B 2	Screenshot of the decrypted file (including the command-line to decrypt): cat decrypted2a.txt

```
User@Keertennah MINGW64 ~
$ cd "/d/master/SEM 2/CRYPTOGRAPHIC ENGINEERING/ciphertexts"

User@Keertennah MINGW64 /d/master/SEM 2/CRYPTOGRAPHIC ENGINEERING/ciphertexts
$ ls -la *.bin
-rw-r--r-- 1 User 197121   0 May 18 03:41 ciphertext_raw.bin
-rw-r--r-- 1 User 197121 480 May  3  2025 ciphertext10a.bin
-rw-r--r-- 1 User 197121 464 May  3  2025 ciphertext10b.bin
-rw-r--r-- 1 User 197121 480 May  3  2025 ciphertext11a.bin
-rw-r--r-- 1 User 197121 464 May  3  2025 ciphertext11b.bin
-rw-r--r-- 1 User 197121 464 May  3  2025 ciphertext12a.bin
-rw-r--r-- 1 User 197121 480 May  3  2025 ciphertext12b.bin
-rw-r--r-- 1 User 197121 480 May  3  2025 ciphertext13a.bin
-rw-r--r-- 1 User 197121 480 May  3  2025 ciphertext13b.bin
-rw-r--r-- 1 User 197121 576 Dec 16  2024 ciphertext1a.bin
-rw-r--r-- 1 User 197121 560 May  3  2025 ciphertext1b.bin
-rw-r--r-- 1 User 197121 464 Dec 16  2024 ciphertext2a.bin
-rw-r--r-- 1 User 197121 464 May  3  2025 ciphertext2b.bin
-rw-r--r-- 1 User 197121 432 Dec 16  2024 ciphertext3a.bin
-rw-r--r-- 1 User 197121 464 May  3  2025 ciphertext3b.bin
-rw-r--r-- 1 User 197121 432 Dec 16  2024 ciphertext4a.bin
-rw-r--r-- 1 User 197121 416 May  3  2025 ciphertext4b.bin
-rw-r--r-- 1 User 197121 416 Dec 16  2024 ciphertext5a.bin
-rw-r--r-- 1 User 197121 448 May  3  2025 ciphertext5b.bin
-rw-r--r-- 1 User 197121 560 May  3  2025 ciphertext6a.bin
-rw-r--r-- 1 User 197121 544 May  3  2025 ciphertext6b.bin
-rw-r--r-- 1 User 197121 464 May  3  2025 ciphertext7a.bin
-rw-r--r-- 1 User 197121 480 May  3  2025 ciphertext7b.bin
-rw-r--r-- 1 User 197121 464 May  3  2025 ciphertext8a.bin
-rw-r--r-- 1 User 197121 512 May  3  2025 ciphertext8b.bin
-rw-r--r-- 1 User 197121 448 May  3  2025 ciphertext9a.bin
-rw-r--r-- 1 User 197121 432 May  3  2025 ciphertext9b.bin
```



PART C: Encryption and decryption (Key Derivation)	[20 *marks*]
C 1	What is PBKDF2? PBKDF2 is a password-based key derivation function version 2 that allows you to derive a cryptographic key based on a password, as well as an accompanying random salt, and an iteration count. As a result, this function makes brute force and dictionary-based attacks more difficult by deliberately slowing down the computation required to create a new key. Using salt also prevents attackers from using pre-computed rainbow tables, while iterating on the key-generation process will further extend the time to create a key.
C 2	Command-line: openssl enc -aes-256-ecb -e -in decrypted2a.txt -out ciphertext2a-2.bin -pbkdf2 -iter 1000 Screenshot of the command-line, and the enter + verifying phase with your password: 
C 3	Command-line: Screenshot of the encrypted file: hexdump -C ciphertext2a-2.bin

```
User@Keertennah MINGW64 /d/master/SEM 2/CRYPTOGRAPHIC ENGINEERING/ciphertexts
$ hexdump -C ciphertext2a-2.bin
00000000: 5361 6c74 6564 5f5f 0367 494b 651a 471b  Salted__.gIKe.G.
00000010: ca26 6bd9 e926 7bfb a7f6 e178 b196 79ab  .&k..&{....x..y.
00000020: 66ab ae43 2166 1e95 d6a0 9ac2 362c 809c  f..C!f......6,..
00000030: f460 4903 9ceb 5375 7fa9 6782 60db 3394  .`I...Su..g.`.3.
00000040: 539b 4bc0 4d04 1dc6 b2eb 86b1 6ba4 a746  S.K.M.......k..F
00000050: 59cb 5363 5617 ac8c 757f e3a2 bfaa 528e  Y.ScV...u.....R.
00000060: d666 614d 2d68 43cb 166c 533a b011 590d  .faM-hC..lS:..Y.
00000070: c4c4 0e00 b96a 27fb 151c 5dc7 1fd9 8c38  .....j'...]....8
00000080: 569e 4dd1 93c7 2a30 90f5 5791 d697 7243  V.M...*0..W...rC
00000090: 4992 0851 dec4 722a 75bc 5307 5748 feea  I..Q..r*u.S.WH..
000000a0: f250 3489 c343 c89d 593e 43dd df96 d83f  .P4..C..Y>C....?
000000b0: 4c96 a7ca 5d59 6ef0 ebb2 b5f2 8de0 0a90  L...]Yn.........
000000c0: 6380 5012 1cca 6bc3 434d 6031 1d89 a629  c.P...k.CM`1...)
000000d0: 3f48 11b5 1a12 e371 a9b5 c44f 8625 e767  ?H.....q...O.%.g
000000e0: 7b9e c170 712c 2ebc d098 923a 1627 493e  {..pq,.....:.'I>
000000f0: f0ea ac40 c156 fdfd c234 cb93 aea1 52db  ...@.V...4....R.
00000100: 44a9 ca5f 5006 0ef8 dba4 a9c8 2e5c bc60  D._P........\.`
00000110: adbe e865 36d6 e768 3ae9 c5fc 7897 6b37  ...e6..h:...x.k7
00000120: 6032 7943 a38d 49c1 4746 fee9 b9dc 7313  `2yC..I.GF....s.
00000130: 5adc dc42 5665 f321 e3cc 2e37 e780 4c4b  Z..BVe.!...7..LK
00000140: 4396 57ff c4cf 32aa 243c e979 330a 9e28  C.W...2.$<.y3..(
00000150: 15bb e45c dafc c26e 3cae 2f3c 8019 35e6  ...\...n<./<..5.
00000160: 4224 2efd 54cb 72f0 4c02 57ce 0b5c 4b19  B$..T.r.L.W..\K.
00000170: 51e6 e53b ff1b ba4f 65cd b98a b823 f652  Q..;...Oe....#.R
00000180: 3a25 97dc ec21 d1e5 8508 90d5 2a42 95f6  :%...!......*B..
00000190: 24ac 0479 b355 be34 9194 912c 6ea3 6d72  $..y.U.4...,n.mr
000001a0: 201e 4a90 057f 48b4 3c12 3e36 e47a 5b7f   .J...H.<.>6.z[.
000001b0: cae3 6c25 e31b fd5f 5609 ceda cb0e 9db9  ..l%..._V.......
000001c0: aafe 73bd ebdf 7145 6f5e 88fc 14d4 01e7  ..s...qEo^......
000001d0: a456 2821 dcd2 ca0f 2bd1 ffd0 cae0 d138  .V(!....+......8
```

C4

Command-line:

openssl enc -aes-256-ecb -d -in ciphertext2a-2.bin -out decrypted2a-2.txt -pbkdf2 -iter 1000

```
User@Keertennah MINGW64 /d/master/SEM 2/CRYPTOGRAPHIC ENGINEERING/ciphertexts
$ openssl enc -aes-256-ecb -d -in ciphertext2a-2.bin -out decrypted2a-2.txt -pbkdf2 -iter
enter AES-256-ECB decryption password:
```

C5

Command-line:

openssl enc -aes-256-ecb -e -in decrypted2a.txt -out temp.bin -pbkdf2 -iter 1000 -p

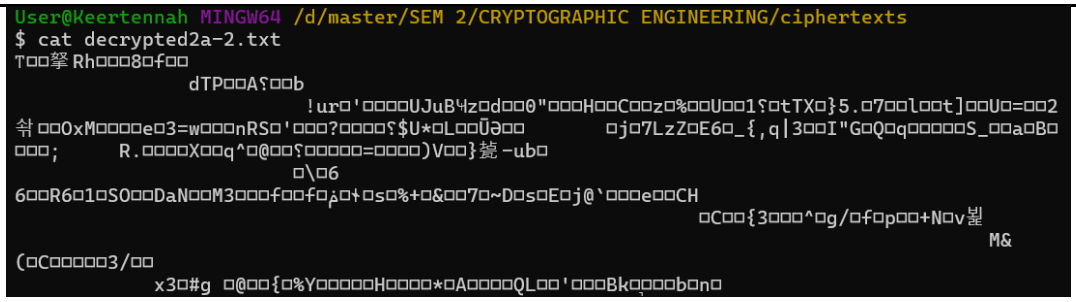
Screenshot those values:

```
User@Keertennah MINGW64 /d/master/SEM 2/CRYPTOGRAPHIC ENGINEERING/ciphertexts
$ openssl enc -aes-256-ecb -e -in decrypted2a.txt -out temp.bin -pbkdf2 -iter 1000 -p
enter AES-256-ECB encryption password:

Verifying - enter AES-256-ECB encryption password:

salt=B1D647BC65388C04
key=FB0F3EDFAC029B640635F4899C8A1AAB6B7B394D0EE89CADD4263138902AAC9E
```

C6

Screenshot of the decrypted file (including the command-line to decrypt):

cat decrypted2a-2.txt



C 7

Briefly describe

PBKDF2 is a key stretching function that has three important security properties which are salt, iteration count and key stretching. Salt is a random number added to the password to ensure that identical passwords result in different keys for every user or session. This prevents attackers from using rainbow tables to look up the hashed value associated with a given password. An increase in the number of iterations will greatly increase the computational cost of each derived key and therefore will slow down brute-force and dictionary attacks on the password. Key stretching by generating a longer key from a low-entropy password will make it very expensive to perform an exhaustive search for the password.